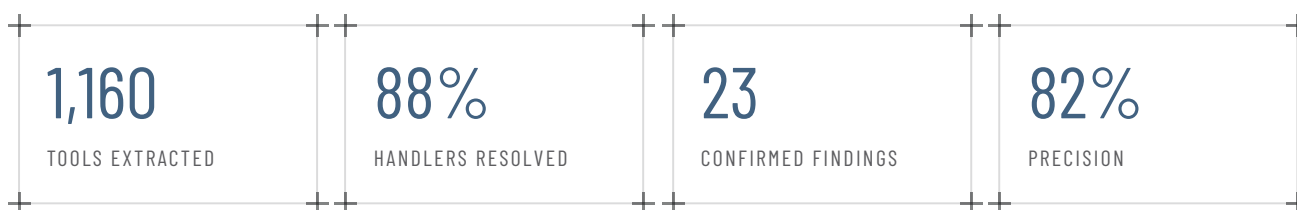


DON'T TRUST MCP HINT DECLARATIONS

We scanned 20 public MCP servers. Twenty-three tools contradicted their own annotations.

MCP tool annotations are a safety contract: `destructiveHint` tells a client that a tool needs human approval before it runs. HintLint checks whether that contract matches the code underneath it. Across 20 source-available servers we confirmed 23 mismatches at 82% precision – most of them tools that delete cloud resources while declaring nothing at all.



WHAT WE CHECKED – AND WHAT WE ARE NOT CLAIMING

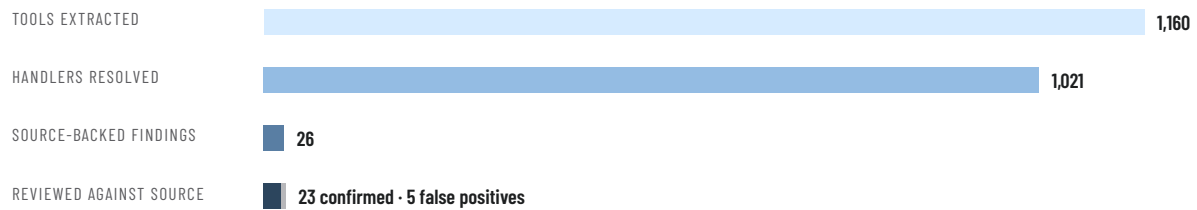
We selected 20 source-available MCP server repositories from GitHub on 2026-07-30 by star count and MCP registry presence, limited to TypeScript, JavaScript and Python – Go, C#, Rust and Java have no extractor support yet. HintLint 0.1.0 ran with the Semgrep Docker evidence engine.

Every finding was then re-verified against source automatically: the checker reopens the file, confirms the reported sink exists at the evidence line, and checks which annotations are actually declared. Bandit was run over all 20 repositories via Docker as independent corroboration.

CLAIM LEVEL: CURATED PILOT

This is not a random sample. It does not support ecosystem-wide prevalence claims, and nothing here should be read as a rate for MCP servers in general – only as what we found in these 20 repositories, at these commits, on this date.

FIG. 1 – FROM 1,160 TOOLS TO 23 CONFIRMED FINDINGS



Each stage is a narrowing: only tools whose handler body could be resolved can be checked against real sinks, and only findings that survive automated source re-verification are reported. 82% of source-backed findings held up.

FINDING 1 – DESTRUCTIVE CLOUD OPERATIONS WITH NO DESTRUCTIVEHINT

20 findings

2 servers affected

Severity: high

Twenty tools call destructive cloud APIs – AWS IAM `delete_access_key`, ElastiCache `delete_cache_cluster`, HealthImaging `delete_image_set`, Timestream `delete_db_instance` and others – without declaring `destructiveHint=true`.

Why it matters: a client that uses annotations for policy enforcement reads these tools as non-destructive and skips human approval before executing an irreversible delete.

```
@mcp.tool() # No annotations declared
async def delete_access_key(user_name: str, access_key_id: str, confirmed:
bool):
    # ...
    iam.delete_access_key(UserName=user_name, AccessKeyId=access_key_id)
```

Fig. 2 – HintLint reads the `delete_access_key()` call as `cloud_mutation` evidence, then flags the decorator above it for declaring nothing.

Suggested fix: add `annotations={"destructiveHint": True, "readOnlyHint": False}` to the `@mcp.tool()` decorator.

FINDING 2 – UNSAFE URL CONSTRUCTION FROM TOOL INPUT

1 finding

1 server affected

Severity: high

One tool builds an outbound URL from user-supplied input with no recognized allowlist or validation; the input flows into a `fetch()` that reaches an external API. If that input is shaped by an LLM prompt injection, the request can be pointed at an attacker-controlled endpoint — server-side request forgery with the server's own network position.

THE OTHER FAILURE MODE: TOOLS THAT DECLARE THE OPPOSITE

Missing annotations are only half the problem. A tool can also declare `readOnlyHint=true` while issuing writes — and that is the worse case, because a client reads it not as unknown but as positively safe. HintLint checks for it (`HINTLINT-READONLY-001`), and the fixture harness catches it on tools that POST or run mutating queries behind a read-only claim.

In this public sample, two such candidates were raised and **both turned out to be false positives**: the evidence had been attributed to a neighbouring tool in the same file. So we confirm no false declarations here — but we also can't clear the sample of them, because a false declaration is only visible where the handler resolves, and 139 tools never did.

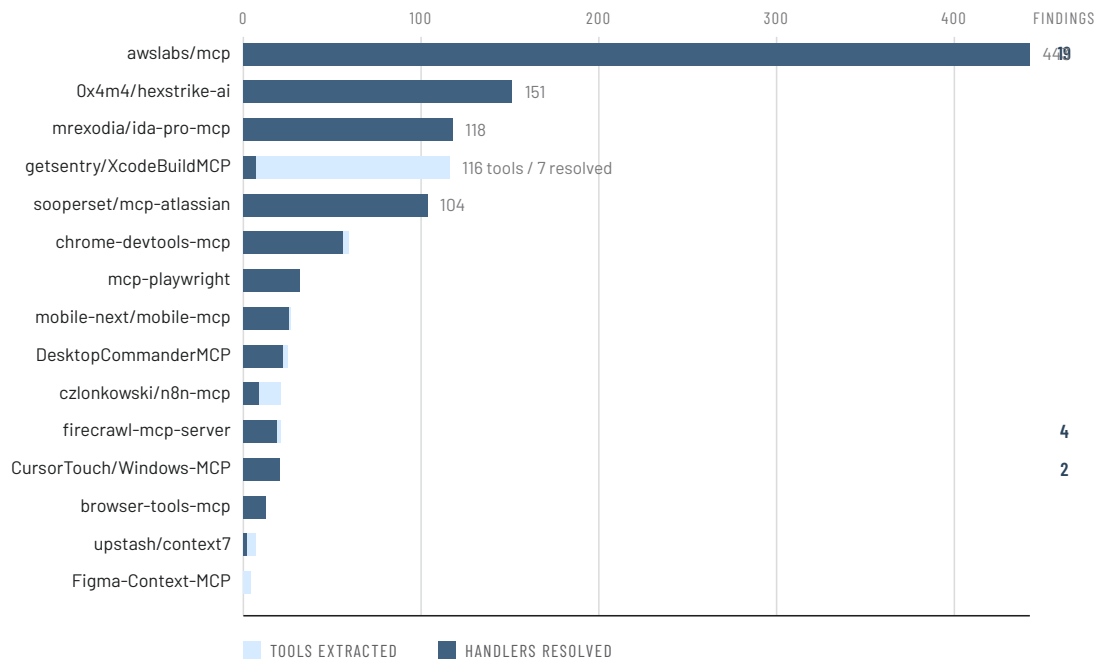
THE OPERATIVE RULE

Absent annotations should be treated as *unknown*, not safe. Present annotations should be treated as *claims*, not evidence. Both need verification against source before a policy engine acts on them.

WHERE THE FINDINGS LANDED

Findings concentrate hard. Every finding in the scan comes from three of the twenty repositories, and the largest of them — a monorepo of 443 AWS tools — carries 19 on its own. Size is not the driver: several repositories with a hundred-plus tools came back clean.

FIG. 3 – TOOLS EXTRACTED VS. HANDLERS RESOLVED, BY REPOSITORY



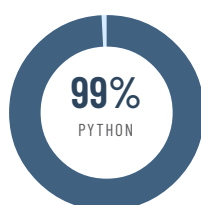
Five repositories in the sample exposed no statically extractable tools at all and are omitted. Where the pale bar runs past the solid one, tools exist that HintLint could see but not follow into a handler body – those tools are unchecked, not clean.

LANGUAGE COVERAGE IS THE LIMITING FACTOR

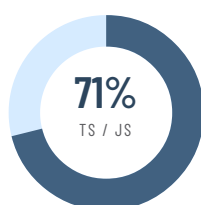
Python servers resolve almost perfectly: decorator-based tool definitions sit next to their handler bodies. TypeScript servers are harder – tools are registered through factories, dynamic manifests and imported arrays, and resolution drops to 71%.

FIG. 4 – HANDLER RESOLUTION BY LANGUAGE, AND WHAT THE 139 UNRESOLVED TOOLS ARE

HANDLER RESOLUTION RATE

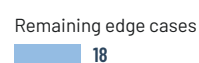


7 repos · 836 tools



13 repos · 324 tools

139 UNRESOLVED TOOLS



Dynamically loaded tools need runtime `tools/list` introspection cross-file import resolution is incomplete.

KNOWN COVERAGE GAPS

The gaps below are documented limits of the current scanner, not clean results. A tool that falls into any of them is **unchecked** – treat it as unknown risk rather than as a passing tool.

GAP	WHY IT ISN'T COVERED	RESOLUTION
Dynamically loaded tools	Tools registered at runtime never appear in source	Requires runtime <code>tools/list</code> introspection
Imported tool arrays	Definitions live in another module	Cross-file resolution incomplete
Go, Rust, Java, C#	No extractor for these languages	Not currently supported
Reflection / generated handlers	Handler body doesn't exist statically	May not resolve statically
Runtime-only behaviour	External API responses, feature flags, environment-dependent branches	Out of reach of static analysis
L4 runtime verification	Live tool behaviour vs. declared annotations	Documented, not yet implemented

THE FIVE WE GOT WRONG

Five findings did not survive review. Two of them claimed a tool declared `readOnlyHint=true` while issuing HTTP POSTs; the evidence had in fact been attributed to a neighbouring tool in the same file – a handler-scope resolution bug, not a real mismatch.

Publishing the false positives and their causes is part of the point: a scanner whose error modes are unknown cannot be trusted to gate anything.

ROOT CAUSE	COUNT	STATUS
Handler scope resolution – evidence attributed to adjacent tool	2	Extractor bug identified
Python <code>ToolAnnotations()</code> constructor not parsed	2	Fixed in 0.1.0
Generic sink label doesn't match actual source text	1	Labeling improvement planned

All five have identified root causes. Three are fixed in the current version; two await handler-scope resolution work.

THE FULL SAMPLE

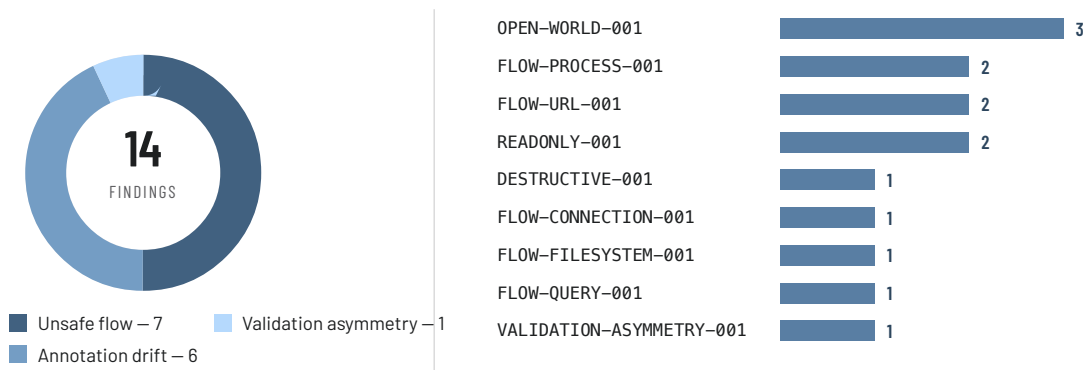
All repositories were public and source-available at scan time; exact commit hashes are recorded in scan metadata.

REPOSITORY	LANGUAGE	TOOLS	HANDLERS	FINDINGS
awslabs/mcp	Python	443	443	19
0x4m4/hexstrike-ai	Python	151	151	0
mrexodia/ida-pro-mcp	Python	118	118	0
getsentry/XcodeBuildMCP	TypeScript	116	7	0
sooperset/mcp-atlassian	Python	104	104	0
ChromeDevTools/chrome-devtools-mcp	TypeScript	59	56	0
executeautomation/mcp-playwright	TypeScript	32	32	0
mobile-next/mobile-mcp	TypeScript	27	26	0
wonderwhy-er/DesktopCommanderMCP	TypeScript	25	22	0
czlonkowski/n8n-mcp	TypeScript	21	9	0
firecrawl/firecrawl-mcp-server	TypeScript	21	19	4
CursorTouch/Windows-MCP	Python	20	20	2
AgentDeskAI/browser-tools-mcp	JavaScript	12	12	0
upstash/context7	TypeScript	7	2	0
GLips/Figma-Context-MCP	TypeScript	4	0	0
microsoft/playwright-mcp	TypeScript	0	0	0
oraios/serena	Multi	0	0	0
BrowserMCP/mcp	TypeScript	0	0	0
epiral/bb-browser	TypeScript	0	0	0
jacob-bd/gemini-notebook-mcp-cli	Python	0	0	0

APPENDIX – WHAT THE RULE SET CATCHES ON FIXTURES

The public scan exercises only part of the rule set. A checked-in fixture harness – four small servers, 17 tools, run with `node scripts/scan-benchmark.js` – covers the rest, and shows the shape of the checks HintLint makes. These are local fixtures, not an ecosystem sample.

FIG. 5 – 14 SOURCE-BACKED FIXTURE FINDINGS, BY CLASS AND BY RULE



Four fixture servers, 17 tools, 15 source evidence records, 14 findings – all 14 source-backed. Critical cases include shelling out to `az` with tool input (`server.py:21`) and an unsanitised Postgres query that also declares `readOnlyHint`.

DISCLOSURE, AND HOW TO REPRODUCE THIS

No upstream issues or pull requests have been opened from this scan. Findings are published here as aggregate data for the MCP ecosystem; individual maintainer notifications, with evidence packets and suggested patches, will follow responsible-disclosure timelines.

```
git clone https://github.com/complira/hintlint.git
cd hintlint
npm run benchmark:public -- --skip-fetch --semgrep docker
npm run review:auto
npm run review:report
```

Raw scan data, review templates and validated reports are in `benchmark/results-public/`.

SOURCE	PROJECT SITE	DOCS
github.com/complira/hintlint	complira.github.io/hintlint	

Disclaimer. This report reflects findings at a specific point in time; repositories may have been updated since the scan date. Findings come from static source analysis and automated review – they are not confirmed exploits. The curated sample does not represent the full MCP ecosystem and should not be used for prevalence claims beyond the 20 repositories listed. HintLint 0.1.0.